The Algebraic Blind Spot in Pattern-Matching Defenses

Joseph Robert Lopez

May 2026

Contents

The Algebraic Blind Spot in Pattern-Matching Defenses

Joseph Robert Lopez

The argument in one paragraph

Substring-matching pattern tiers — regex WAFs, spam filters, secret scanners, LLM guardrails — share an algebraic property that determines what they can and cannot see, independent of how their patterns are written. The property is aperiodicity of the syntactic monoid, and its consequence is structural: no pattern in this class can decide whether a count is even or odd. An attacker who hides a payload across alternating positions — payload on even indices, filler on odd — bypasses the tier, because deciding the alternation is precisely the modular-counting predicate the tier provably cannot compute. This article reports on a measurement of 142 production patterns drawn from twelve sources (100% aperiodic, 100% bypass under MOD_2 encoding), establishes that no composition of pattern matchers closes the bypass class, and identifies the architectural change that does.

Who this is for, and why now

This is for security architects responsible for input-filtering pipelines: WAFs, spam filters, code and secret scanners, IDS regex tiers, and LLM guardrails. The shared structural property cuts across

all of them, and the implication for defense-in-depth composition is concrete enough to act on this quarter. Russinovich et al. recently framed LLM jailbreaks as a surmountable challenge addressable through layered defense [11]. We complement that framing: one specific layer — substring matching — is structurally unsurmountable, and the implication for composition is that serial-AND pipelines do not recover from this layer's blindness, regardless of how capable the downstream tier is.

The result is not new mathematics. It chains three classical theorems — Schützenberger (1965) [13], Barrington-Compton-Straubing-Thérien (1992) [1], and Furst-Saxe-Sipser (1981) [4] — onto an empirical security setting. The novelty is the diagnostic, the measurement, and the architectural conclusion.

The bypass

Take any substring-matching pattern. The canonical example from our corpus is `exec\s*\(`, which appears in nine of the twelve sources we surveyed and matches the literal substring `exec(` (with optional whitespace). To bypass it, encode the payload across alternating positions: write `exxxexcx(`, where every other character is filler. The pattern does not match, because no contiguous substring of the encoded string is `exec(`. The decoded payload — read every other character starting at index 0 — is `exec(`. The decoder is a parity operation. The pattern cannot perform parity.

This is the entire attack. It generalizes from MOD_2 (alternating positions) to MOD_p for any prime p (every p-th position is payload, the others are filler). It absorbs several documented evasion techniques as instances:

- **Reading every other character** is MOD_2 with arbitrary filler.
- **Acrostics** are MOD_k at line granularity rather than character granularity.
- **Zero-width Unicode insertion** (U+200B, U+200C) between payload characters is MOD_2 with the null filler.
- **Token splitting** that forces the tokenizer to reassemble a keyword across boundaries is a partial MOD_2 at token level.

The reason these all work, and the reason no further pattern can be added to the tier to close them, is the same: the substring-matching design pattern compiles to an aperiodic finite-state machine, and aperiodic machines provably cannot count modulo any prime.

Why this is structural

The chain has four steps and is unconditional in the regex setting:

Sidebar: The four-line chain

1. Substring-matching languages of the form `$\Sigma$* · L(r) · $\Sigma$*` are **star-free**: they are definable from Σ, , $\{\varepsilon\}$, and single letters by Boolean operations and concatenation, without Kleene star (Theorem 6.1 of the companion preprint, by case analysis over the regex grammar).
2. Star-free languages have **aperiodic syntactic monoids** (Schützenberger 1965 [13], McNaughton-Papert 1971 [9]).
3. Aperiodic regular languages are exactly those in $\mathbf{AC}^0$ (Barrington-Compton-Straubing-Thérien 1992 [1]).
4. AC^0 cannot compute MOD_p for any prime $p \geq 2$ (Furst-Saxe-Sipser 1981 [4]; Håstad 1987 [6]).

Conclusion: every substring-matching pattern is in AC^0, hence cannot decide MOD_p, hence is blind to any payload encoding whose decoder is a modular-counting operation. The conclusion holds for any finite Boolean composition of such patterns: star-free is closed under union, intersection, and complement, so stacking matchers preserves the blindness.

This is the load-bearing argument. The patchability question follows immediately: any further substring-matching pattern added to the tier is itself star-free, the union remains star-free, the syntactic monoid remains aperiodic, and the new tier remains in AC^0. The MOD_p bypass class is not closed by any finite addition of substring-matching patterns. This is a closure-under-composition statement, not a tuning recommendation.

The same argument extends to the broader class of star-free string matchers: tries, finite DFA blocklists, Bloom filters on their intended literal sets. Every one of these recognizes a star-free language by construction; every one is in AC^0; every Boolean composition of them remains in AC^0. The architectural consequence is that no defense-in-depth pipeline composed entirely of substring-matching tiers can close the modular-counting bypass class, regardless of how the tiers are written, ordered, or weighted.

The corpus

To turn the structural argument into a measurement that practitioners can act on, we collected 142 regex patterns from twelve sources and ran them through a monoid extractor. The breakdown:

- **Nine third-party open-source guardrail projects** contributed 100 patterns: LLM-Guard, llm-guard-py, Rebuff, Guardrails-AI, Presidio, GitLeaks, LangKit, BodAIGuard, and the regex subset of NeMo Guardrails' content rails.
- **Three author-assembled sets** contributed 42 patterns: 8 inspired by the OWASP Top 10 for LLMs taxonomy, 15 from a curated WAF-generic pattern set, and 19 author-constructed adversarial test patterns.

For each pattern r, we compiled to a Thompson NFA, converted to the minimal DFA via Hopcroft's algorithm, enumerated the transition monoid by BFS over the Cayley graph, and tested aperiodicity directly: for each monoid element m, check whether the sequence m, m^2, m^3, … stabilizes (i.e., m^n = m^{n+1} for some computable $n \leq |M(L\_r)|^2$).

Result. All 142 patterns (100%) have aperiodic syntactic monoids. All 142 admit the MOD_2 bypass: for each pattern we constructed an encoded payload string $\phi_2(h)$ where h matched the pattern in plaintext, and verified the pattern did not match $\phi_2(h)$.

Table 1 shows representative patterns with their DFA size, monoid size, and aperiodicity verdict. The monoid extractor is released as an artifact of this work (~860 lines of Python; Zenodo DOI in the artifact bundle).

ID	Pattern	DFA states	|M|	Aperiodic
F1	`rm\s*-[rR][fF]`	9	18	Y
F2	`wget|curl.*malware`	14	31	Y
F3	`import\s+os`	11	22	Y
F4	`eval\s*\(`	8	14	Y
F5	`exec\s*\(`	8	14	Y

Table 1. Representative patterns from the 142-pattern corpus. DFA states and monoid sizes are for the substring-matching wrap `$\Sigma$* · L(r) · $\Sigma$*`. All 142 patterns in the full corpus are aperiodic.

Beyond LLM guardrails: WAFs and spam filters

The corpus measurement establishes the result for the LLM-guardrail class. To extend the empirical leg to two non-LLM defensive classes, we ran the same MOD_2 bypass against six ModSecurity OWASP CRS 4.27 SQL-injection patterns and six SpamAssassin `20_phrases.cf` patterns: 12 patterns total, 20 synthesized payloads (10 per class), three filler choices (NULL byte, ASCII space, U+200B zero-width space).

ModSecurity: 10/10 baseline-matched. NULL bypass: 8/10. ASCII-space: 10/10. ZWSP: 10/10. The two NULL-byte failures are SQL-comment patterns whose alphabet includes `\x00` by design (CRS catches NULL injection explicitly); both bypass under printable-space and zero-width fillers, exactly as the algebraic argument predicts — no single-filler patch closes the class.

SpamAssassin: 9/10 baseline-matched (one synthesizer artifact). NULL: 9/9. ASCII-space: 9/9. ZWSP: 9/9.

Across twelve patterns and twenty payloads from two defensive classes distinct from LLM guardrails, every baseline-matched payload bypassed under at least one of the three filler choices. The classes named in published surveys but not yet measured here — literal-string AV signatures, IDS regex tiers, code/secret scanners — remain by algebraic argument alone; the unification claim is empirical for three classes (LLM guardrails, WAF, spam filter) and structural for the rest.

Composition does not save you

The natural defender's intuition is that stacking tiers covers the gaps: regex first-pass, neural classifier second-pass, output monitor third-pass. The intuition is wrong about what stacking does to the algebraic blindness. The composition behavior depends on whether tiers are combined in serial-AND, parallel-OR, or as a soft-score blend, and on whether the downstream tier has the algebraic capacity the upstream tier lacks. We summarize the five composition cases that matter:

Composition	Blindness preserved?	Why
Regex $\wedge$ Regex (serial)	Yes	Star-free closed under intersection
Regex $\vee$ Regex (parallel)	Yes	Star-free closed under union
Regex $\wedge$ Neural (regex confirms first)	**Yes**	Regex vetoes; if regex misses, the encoded payload is allowed regardless of the neural tier
Regex $\vee$ Neural (parallel-OR)	No (capacity)	Neural tier in TC^0 can compute MOD_p; composed system blocks if either fires
Soft-score blend, weight α on regex	Yes if $\alpha > \alpha^*$	Critical weight $\alpha^* = 1 - \tau/c$; above α^* the regex weight dominates, reimporting AC^0 blindness

Table 2. Composition laws for substring-matching and neural tiers. Items 1–2 follow from Boolean closure of star-free languages. Item 3 — the regex-confirms-first design — propagates the regex layer's blindness because the dangerous case is precisely the one the regex does not flag. Item 4 is the only composition that breaks the ceiling, subject to the realization caveat below. Item 5 quantifies how blends inherit blindness above a critical weight.

The critical row is the third. A "regex confirms, neural inspects" architecture inherits the regex layer's blindness: the encoded payload is invisible to the regex, the regex returns `allow`, the AND short-circuits, and the neural tier's capacity is irrelevant to the outcome. This is the design pattern most commonly deployed in production guardrail stacks — a regex pre-filter feeding a neural classifier, gated by the regex result. The composition trap is that this stack inherits the regex tier's AC^0 blindness, regardless of how capable the neural tier is.

The fourth row — parallel-OR with a TC^0 tier — is the architectural fix, and it carries one important caveat. The capacity statement is that some TC^0 circuit can compute MOD_p; a single uniform attention head computes parity directly via threshold gates [10, 12]. The caveat, due to Hahn (2020) [5], is that fixed-architecture self-attention cannot model parity for unboundedly long inputs unless depth or attention heads scale with sequence length. **Capacity is what the complexity class permits; realization is what the deployed model achieves.**

We validated the realization in the small-payload regime relevant to deployed guardrails. A minimal transformer classifier (one layer, one attention head, d_model=32, 14,753 parameters) trained on character-level MOD_2-encoded payloads achieved 90% detection on held-out MOD_2, 90% on unseen MOD_3, and 100% on MOD_5 and MOD_7, with 0% false-positive rate on clean text. Regex detection across all four encodings was 0%. The capacity is achievable in the deployment-relevant regime, even if the unbounded-length statement remains conditional on architectural scaling.

End-to-end against a production library

To validate the composition argument against a deployed system rather than synthetic patterns, we tested the full defense stack against 23 patterns from LLM-Guard's BanSubstrings scanner — a production guardrail library shipped by ProtectAI. We ran 20 payloads spanning prompt injection, code injection, SQLi, and XSS through four defense configurations and seven encoding types.

Encoding	Regex only	+ Preprocessing	+ Neural	Full stack
Raw payload	95%	95%	95%	95%
MOD_2	**0%**	0%	100%	100%
MOD_3	**0%**	0%	100%	100%
Base64	0%	95%	100%	100%
ROT13	0%	95%	90%	100%
ZWSP	0%	95%	0%	95%
Homoglyph	15%	95%	60%	95%
Leetspeak	0%	95%	80%	100%

Table 3. End-to-end detection against 23 LLM-Guard production patterns. "Regex Only" is the current default configuration. "Full Stack" adds preprocessing (strip, decode, normalize) and a parallel neural tier.

The "Regex Only" column is the algebraic blindness theorem made operational: the production library detects **0% of encoded payloads across all seven encoding types**. The "Full Stack" column shows 95–100% detection across the same encodings. The defense gap is entirely attributable to the AC^0 ceiling; the recovery is entirely attributable to ceiling-breaking composition (parallel-OR with a TC^0 tier) plus invertibility-based preprocessing.

An architectural checklist

The diagnostic above produces a short list of architectural questions that determine whether a given pipeline is exposed to this class of bypass. These are review questions, not vendor-cornering questions; they are the ones we would ask of our own systems before asking them of someone else's.

1. What is the algebraic class of each tier? A pattern-matching tier (regex, trie, finite DFA blocklist, Bloom filter on a literal set) is in AC^0 and is provably MOD_p-blind. A neural classifier in the transformer family is in TC^0 and has the capacity, though not necessarily the realized behavior, to detect MOD_p. The operational question is: which tiers are in AC^0, which are in TC^0, and where does the boundary fall in the pipeline?

2. What is the composition topology? Serial-AND (regex confirms first, neural inspects after) propagates the upstream tier's blindness. Parallel-OR (block if either fires) recovers the downstream tier's capacity. Soft-score blends require the regex weight to stay below a critical $\alpha^* = 1 - \tau/c$; above that weight, the blend reimports the regex tier's blindness. Pipelines that look like "regex → neural" with the AND semantics are the common high-risk case.

3. What encodings does preprocessing handle, and which are out of scope? Invertible encodings (Base64, ROT13, common Unicode normalizations) can be undone before the pattern tier sees the input; they should be. Modular-counting encodings (MOD_p, ZWSP insertion, alternating-position payloads) are not invertible without the modulus and cannot be enumerated by preprocessing alone; they require the TC^0 tier in parallel, not preprocessing.

4. Is the audit theorem-backed or heuristic? The monoid extractor releases a constructive audit: given a pattern, it computes the minimal DFA, enumerates the transition monoid, identifies all group components via standard semigroup algorithms, and outputs the complete blindness spectrum — the set of primes p for which the pattern is provably MOD_p-blind (Krohn-Rhodes decomposition [8]; Proposition 6.11 of the preprint). For aperiodic patterns the spectrum is all primes. A clean audit report is a mathematical proof of the stated blindness, not a confidence-based assertion. Integration is straightforward in CI/CD as a pre-commit check; analysis of the 142-pattern corpus completes in under 60 seconds.

5. What lives at the execution layer? A class of attacks — semantic and homomorphic-reasoning attacks (Sections 7.3–7.4 of the preprint, demonstrated empirically) — cannot be defended at any inference layer, regardless of the algebraic class of the tiers. These attacks operate by having the LLM solve an abstract grammar over opaque symbols, with the interpretation table held offline by the attacker. The LLM-visible content is a legitimate-looking formal-reasoning exercise; the harmful instantiation happens after the model's output passes back through the attacker's interpretation table. Defense for this class requires monitoring at the execution layer — what the LLM does, not what it receives — and is the natural complement to inference-layer pattern matching.

The five questions are not exhaustive, and the answers will be specific to each deployment. But they are concrete, theorem-backed, and answerable from the pipeline's configuration plus a sixty-second monoid-extractor run. That is the level of architectural review the algebraic blindness result

enables and, in our reading, requires.

What this does not solve

The algebraic blindness result is unconditional for substring-matching tiers. The story is more nuanced one tier up. Neural guardrails in the transformer family escape AC^0 and the modular-counting bypass — that is the content of the parallel-OR recovery. They face their own structural barriers, which the algebraic result does not address.

First, TC^0 itself has a conjectured ceiling at NC^1. Problems that require evaluating arbitrary Boolean formulas over input tokens are believed to lie outside TC^0, though the separation has resisted proof for four decades. A guardrail that must evaluate a complex logical safety policy over its input is conjectured to require NC^1 or higher; attackers who can force the detection problem into the NC^1 TC^0 gap can — conditionally on the conjectured separation — evade neural guardrails by an analogous complexity argument. The unconditional result for regex does not transfer to a corresponding unconditional result for neural tiers.

Second, neural tiers are subject to gradient-based adversarial attacks [14] that operate in weight space rather than in the language's algebraic structure. This is a parallel threat model, not a counter-argument to the composition recommendation, but it is a real cost to be priced in. Three to four orders of magnitude more compute than DFA traversal is the typical figure.

Third, the homomorphic-reasoning class of attacks (V3/V4 in the preprint) operates by abstraction rather than by encoding. The LLM-visible prompt contains an abstract grammar over opaque symbols and is, as a string, indistinguishable from a legitimate formal-reasoning exercise. No content-inspection tier — regex, neural, or any composition — can distinguish the adversarial prompt from a legitimate one, because at the prompt boundary they are the same prompt. The defense for this class is execution-layer, not inference-layer, and the architectural checklist's fifth question is the entry point.

The intent of the algebraic result is not to claim that pattern matching is useless — it catches what it was designed to catch, the threats whose distinguishing feature is a contiguous substring. The intent is to mark the boundary of what the design can do, so that pipelines built on top of it do not over-claim coverage that the upstream tier provably cannot provide.

Closing

The substring-matching tier is doing real work, and we are not arguing for its removal. We are arguing for two architectural changes that follow from a structural property of the tier, not from any criticism of the patterns it ships with. First, do not compose pattern-matching tiers serially with a more-capable downstream tier and call the composition defense-in-depth. Serial-AND propagates the upstream tier's blindness; parallel-OR recovers the downstream tier's capacity; soft-score blends require the regex weight to stay below a critical threshold. Second, audit the algebraic class of each tier and record its blindness spectrum. The audit is constructive, theorem-backed, and runs in seconds.

This complements rather than contradicts Russinovich et al.'s surmountable challenge framing [11]. Their argument is that LLM jailbreak defense is winnable through layered defense; ours is that one specific layer is structurally unsurmountable, and the implication for composition is that some compositions are not layered defense at all but are inherited blindness with extra steps. Both

readings are true. The architectural conclusion that ties them together is that **defense-in-depth is a property of the composition topology, not of the count of tiers.**

Notes and references

The companion technical preprint contains the formal apparatus: full proofs of the substring-aperiodicity theorem and the modular-counting bypass, the Krohn-Rhodes audit construction, the composition laws with full case analysis, and the homomorphic-reasoning attack vectors. The preprint URL, the GitHub repository for the monoid extractor and reproduction scripts, and a Zenodo DOI for the artifact bundle accompany this article.

AI-methodology disclosure. This work was developed using an agentic AI research pipeline; tooling and model usage are documented in the preprint's appendix and the artifact bundle.

References (selected; full list in preprint).

[1] D. A. M. Barrington, K. Compton, H. Straubing, and D. Thérien. Regular languages in NC^1. *Journal of Computer and System Sciences* 44(3), 478–499, 1992.

[2] N. Boucher et al. Bad characters: Imperceptible NLP attacks. *IEEE Symposium on Security and Privacy*, 2022.

[3] D. Chiang, P. Cholak, and A. Pillay. Tighter bounds on the expressivity of transformer encoders. *ICML*, 2023.

[4] M. Furst, J. B. Saxe, and M. Sipser. Parity, circuits, and the polynomial-time hierarchy. *FOCS*, 260–270, 1981.

[5] M. Hahn. Theoretical limitations of self-attention in neural sequence models. *TACL* 8, 156–171, 2020.

[6] J. Håstad. *Computational Limitations of Small-Depth Circuits*. PhD thesis, MIT, 1987.

[7] W. Hackett, L. Birch, S. Trawicki, N. Suri, P. Garraghan. Bypassing LLM guardrails: an empirical analysis. *LLMSEC at ACL*, 2025.

[8] K. Krohn and J. Rhodes. Algebraic theory of machines I. *Trans. AMS* 116, 450–464, 1965.

[9] R. McNaughton and S. Papert. *Counter-Free Automata*. MIT Press, 1971.

[10] W. Merrill and A. Sabharwal. The parallelism tradeoff: limitations of log-precision transformers. *TACL* 11, 531–545, 2023.

[11] M. Russinovich et al. [LLM jailbreak as a surmountable challenge — verify exact citation before submission].

[12] L. Strobl, W. Merrill, G. Weiss, D. Chiang, D. Angluin. Formal language recognition by hard attention transformers. *TACL* 12, 1–19, 2024.

[13] M.-P. Schützenberger. On finite monoids having only trivial subgroups. *Information and Control* 8(2), 190–194, 1965.

[14] A. Zou, Z. Wang, J. Z. Kolter, M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. arXiv:2307.15043, 2023.